GUIA VISUAL 067

AWK
CAMPOS E REGISTROS

*Quanto tráfego cada endereço consumiu no dia? O `cut -d' ' -f1,10` saiu **quase** certo — e "quase" é o problema: ele corta em **cada** espaço, então um espaço duplo desloca tudo, e a requisição entre aspas virou três campos. Pior: o `cut` extrai, mas **não soma**. O `awk` divide sozinho, tem aritmética, tem tabela associativa e tem um lugar para rodar depois da última linha. É a ferramenta entre **"o `grep` não dá conta"** e **"vou escrever em Python"**.*

067
GUIAS LINUX

REGISTRO, CAMPO E O LAÇO IMPLÍCITO



`NF` · `$NF`
Quantos campos tem este registro, e o **último** campo — evita contar coluna na mão e sobrevive a um log com uma coluna a mais.

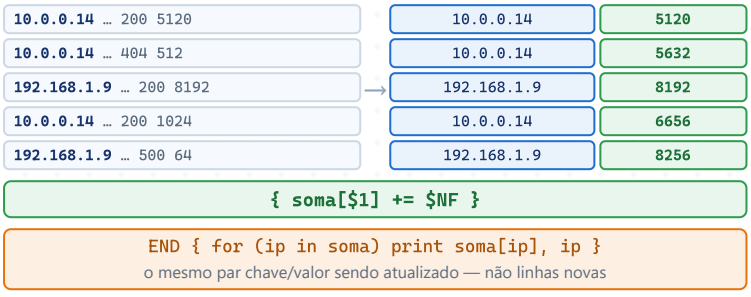
`NR` · `FNR`
Número do registro desde o início; o `FNR` **reinicia a cada arquivo** (e `FILENAME` diz qual é).

`FS` · `OFS`
Como dividir na entrada e como **juntar** na saída. `RS` / `ORS` fazem o mesmo para registros.

padrão `{ ação }`
Padrão sem ação → `print $0`. Ação sem padrão → **roda em toda linha**. Daí `awk '/ERRO/'` ser um `grep` e `awk '{print}'` ser um `cat`.

Repare nos campos em vermelho. A data entre colchetes tem um espaço antes do fuso, então ocupa **dois** campos — e a requisição entre aspas ocupa **três**. Nem o `awk` entende aspas ou colchetes por conta própria: ele divide por espaço em branco e mais nada. **A diferença para o `cut`** é que o `awk` trata *qualquer sequência* de espaços como um separador só, enquanto o `cut` conta cada espaço. Para campos delimitados, mude o `FS` — ele aceita **expressão regular** quando tem mais de um caractere: `-F'[,;]'`, `-F'\t'`, `-F'" '`. Cuidado com um caso especial: `FS=" "` (o padrão) é "sequências de espaço, ignorando as das bordas"; `FS="[ ]"` é "exatamente um espaço". E o `OFS` **só entra em vigor quando `$0` é reconstruído** — daí o idioma obscuro `{ $1=$1; print }`, uma atribuição que não muda nada e força a reconstrução.

AGREGAÇÃO COM ARRAY ASSOCIATIVO — A RAZÃO DE O AWK EXISTIR



Três coisas são automáticas e é por isso que a agregação cabe em duas regras: **o array nasce sozinho**, **chave inexistente vale zero** na primeira soma e o `END` alcança tudo o que sobreviveu ao laço. O custo é **memória**: uma entrada por chave distinta — agregar por endereço num log de milhões de linhas é barato (poucos milhares de chaves); por URL completa com parâmetros pode consumir gigabytes. E **a ordem de `for (k in a)` é arbitrária**: ordene depois, com `sort -rn`.

`awk '!visto[$0]++'` **deduplica preservando a ordem** — o que o `sort -u` não faz. É padrão sem ação (logo, imprime): `visto[$0]++` devolve o valor **antes** de incrementar — zero na primeira aparição (falso, negado vira verdadeiro, imprime) e ≥1 nas seguintes. **E um detalhe que morde:** `if (tab[k] == "")` **cria** a chave como efeito colateral do acesso; para testar sem criar, `if (k in tab)`. `length(arr)` dá o número de chaves e `delete tab[k]` remove uma.

PADRÕES: QUAIS REGISTROS INTERESSAM

`$3 ~ /^ERRO/`	padrão sobre **um campo** (`!~` nega) — não sobre a linha toda
`$9 ≥ 400`	comparação **numérica**: não confunde status com trecho de URL
`NR > 1`	pula o cabeçalho · `NF == 0` pega linha vazia
`BEGIN { }`	antes da 1ª linha: separadores e cabeçalho. Só com `BEGIN`, nem lê a entrada — `awk 'BEGIN{print 3*7}'` é calculadora
`END { }`	depois da última: é onde sai o total

AÇÕES E FUNÇÕES

`total += $9`	variável não se declara e **nova vale zero**
`print $1, $2`	a vírgula insere o `OFS`; **sem vírgula, gruda** (justaposição é concatenação)
`printf "%-15s\n"`	o mesmo do shell, **sem quebra de linha automática**
`substr · split`	índices começam em **1** — a pegadinha de quem alterna com array do Bash
`sub · gsub`	primeira · todas as ocorrências · `match` preenche `RSTART` / `RLENGTH`
`next · exit`	pula o registro · encerra a leitura (**mas ainda roda o `END`**)

FNR==NR: JUNÇÃO DE DOIS ARQUIVOS

`awk 'FNR==NR { ioc[$1]=1; next } $1 in ioc { print "SUSPEITO:", $0 }' iocs.txt acesso.log` — o `FNR==NR` é verdadeiro **apenas durante o primeiro arquivo**, o que permite carregar uma tabela e depois consultá-la. É junção sem banco de dados, e resolve uma classe inteira de problemas que viraria laço aninhado com `grep`. **Qual `awk` está instalado importa:** no Kali/Debian costuma ser o `mawk` (rápido, enxuto); no RHEL é o `gawk`, que tem `gensub()`, `IGNORECASE`, `asort()` e `RS` como expressão regular. Confira com `awk --version`; se o script depende de extensão, o *shebang* é `#!/usr/bin/gawk -f`.

Nunca monte o programa `awk` por interpolação do shell — e cuidado com `system()`. Escrever `awk "\$1 == \"$IP\""` funciona até o dia em que a variável tiver uma aspa: aí o valor **quebra a sintaxe**, e um valor hostil **executa o que quiser**. A forma segura existe para isso: `awk -v alvo="$IP" -v limite=400 '$1 == alvo && $9 ≥ limite'`. Pelo mesmo motivo, `system("…")` e `print | "comando"` montados a partir de dado vindo do alvo — nome de host, *banner* de serviço, cabeçalho HTTP — são **injeção de comando** com o privilégio de quem rodou o `awk`. Se precisa disparar comando por linha, **imprima a lista e passe para o `xargs`**, que não interpreta o conteúdo como shell.